

P04: Makers Makin' It, Act II

Team Name: Orangutans

Roster: Kiran Soemardjo (PM), Mustafa Abdullah (Dev 1), Yu Lu (Dev 2), Eviss Wu (Dev 3)

SHIP DATE: 2026-04-22

Description

We plan to pull data from the Steam API, and use the data to visualize trends in the games that are popular at the moment. The data will also be filterable by rating, player count, genre, tag, etc. Additionally, the user will be able to enter their Steam ID, and get recommendations based on the games in their library, as well as an analysis of personal stats (through the Steam API using the user's ID).

Dependencies

Framework: Tailwind (FrontEnd)

JavaScript: Chart JS

Python: Flask, Sqlite3

APIs: Steam Web API (100k calls/day)

Tasks + Assignments

Project Manager: Kiran Soemardjo

- Flask functionality
- Recommendation algorithm

Dev1: Mustafa Abdullah

- Create database structure
- Data processing/parsing

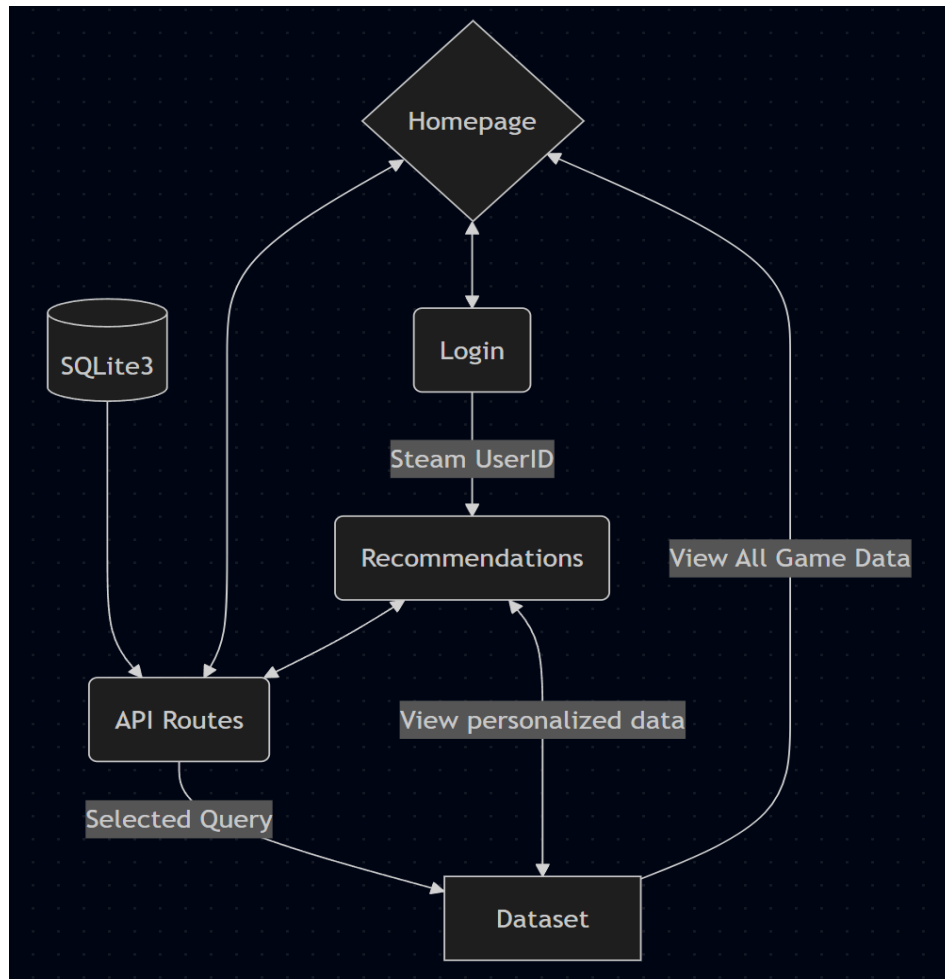
Dev2: Yu Lu

- Implement Javascript for site interactability
- Visualize data with chart.js

Dev3: Eviss Wu

- Frontend Visuals
- UI
- API Calling

Site Map



Routes:

- `/` (homepage)
- `/login` (authentication page where Steam ID is inputted – no registration)
- `/recommendations` (user page from authentication)
- `/recommendations/queue` (generate a new discovery queue based on the current user's data)
- `/filter` (manual filter selection)
- `/dataset` (dataset results visualized, reached through `/`, `/filter`, or `/recommendations`)

Front-End Framework: Tailwind

We initially chose to use bootstrap for its numerous pre-buils, but as we decided that we would create a site that would aggregate data for Steam, we found it better to use tailwind, as it could emulate steam's styling effectively, and also was used initially by Yu. For that reason, we changed to tailwind to support our needs for customization, especially in terms of charts, flash windows, the loading screen, and our carousels, which we did not get to do unfortunately. We found that Tailwind ended up being much more blocky and bloated, however since Steam uses React for its front page, it ended up being less work due to high React to Tailwind compatibility.

Data Visualization Library: chart.js

Due to our data mainly being used to compare games and their monetary success or playercount, we will typically have two axes. This means that a trendline graph, bar graph or pie chart should be enough to suit our needs. We also chose this because we were all familiar with javascript and HTML, which [chart.js](#) blends in order to provide its clean-looking chart. [Chart.js](#) provides built-in interactivity as well, allowing us to do things like filter our charts for specific times, genres, etc. if need be. On the other hand, [chart.js](#) may be slow when it comes to larger datasets and the pre-built charts may look un-unique.

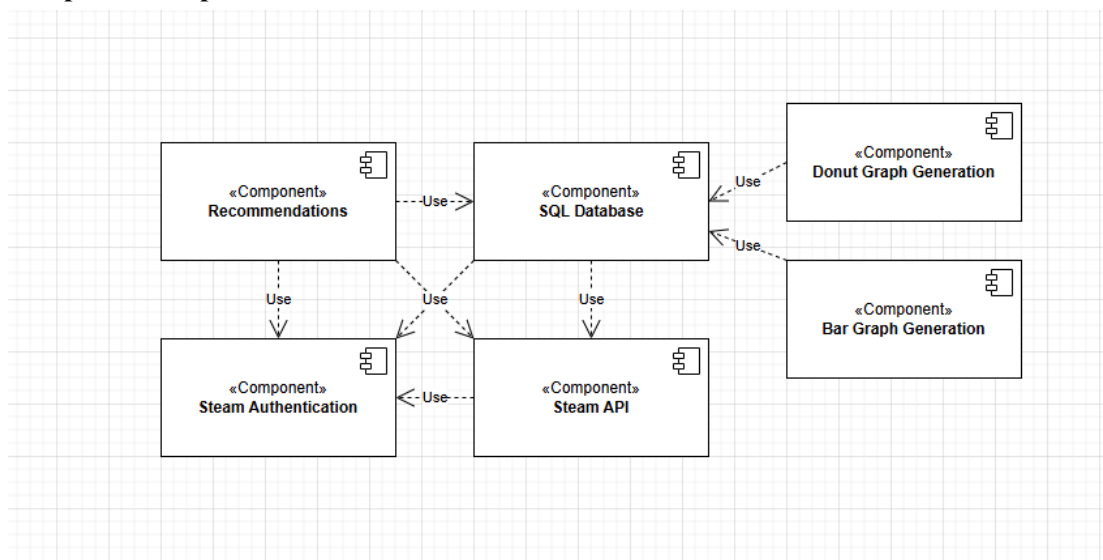
External Resources

<https://github.com/MattRyder/SteamAPI>

Components

- Steam API
 - Methods pulling from raw data and user data
- SQL Database
 - Stores data for easy retrieval for specific queries
- Donut Graph Generation
 - Creates donut graph from filtered dataset for visualizing trends by genre/tag
- Bar Graph Generation
 - Creates bar graph from filtered dataset for visualizing stats by game
- Steam Authentication
 - Authentication of Steam ID, pulling user data from the API for use
- Recommendations
 - Uses user library/playtime data to generate recommendations

Component Map



Database Organization

Games		
TEXT	app_id	PK
TEXT	name	
TEXT	release_date	
TEXT	developer	
TEXT	publisher	
BOOLEAN	is_free	
DOUBLE	price	
TEXT	description	
TEXT	genre_list	
TEXT	tag_list	
TEXT	tag_vector	

Reviews		
INTEGER	app_id	FK REF games(app_id)
INTEGER	review_score	
TEXT	review_score_desc	
INTEGER	total_positive	
INTEGER	total_negative	
INTEGER	total_reviews	

Users		
INTEGER	steam_id	PK
TEXT	games	
TEXT	playtimes	

Player Stats		
INTEGER	app_id	FK REF games(app_id)
INTEGER	current_players	
INTEGER	peak_players_today	
INTEGER	time_fetched	